



Semantic Caching Engine

Problem Statement →



| Each and every business wants to save money!

→ Companies when they use LLM API → Each and every query hits a lot of money to provide a response!



Building a cache for similar response would enhance efficiency and save time

Imagine →

- User A → How do I reset password?
- User B → I forgot my password?



Instead of hitting a LLM query, we could provide the same set of answer cached in our memory to both!

Seven Criteria!

Which I understood in yesterday's meeting and will play a crucial role in building the project!

1. **Latest Dataset** → 2026 LMSYS Chatbot Arena
URL → https://huggingface.co/datasets/lmsys/chatbot_arena_conversations
2. **Better & Efficient Tech** → Caching is an ancient computer science concept. *Semantic* caching using Vector Databases (like MongoDB Atlas Vector Search) and advanced AI embeddings on AWS is cutting-edge.
3. **Tech Stack** →

Backend	Java + Spring Boot
AWS	DynamoDB, EC2/Lambda
GenAI	Embedding models and Vector DB

4. **Efficient Algorithm** → Replacing the Standard Least Recently Used Cache eviction with
Researched and Developed → **ML-based Predictive Eviction Algorithm**—an algorithm that predicts which data to delete from the cache based on semantic usage patterns
 - Less than a 50-line code of JAVA!

$$\text{Score} = w_1 \cdot \text{Recency} + w_2 \cdot \text{Frequency} + w_3 \cdot \text{Semantic Cluster Density}$$

1. **Distribution among team** →

Member 1	AWS Cloud Infrastructure & Networking
Member 2	GenAI Embeddings and Vector DB (MongoDB)
Member 3	Java SpringBoot Backend and API routing
Member 4	Algorithm design & Data Analytics

6. **Research Angle** → A lot to research the paper could even be titled →

✂ A Novel Semantic Caching Architecture for Optimizing Compute Costs and Query Latency in Large Language Models on AWS

- **1. Latency Delta (Time Savings):** The comparison
 - **2. Cloud Token (Cost Savings)**
 - **3. Eviction Algorithm Benchmarking (The Core Research):** Comparing Cache Hit Ratios (%) over time between standard LRU (Least Recently Used) and your custom **Semantic Density Eviction Algorithm**.
 - **4. Cosine Similarity Thresholds vs. Accuracy**
 - **5. System Throughput (RPS):** Benchmarking the Requests Per Second (RPS) the Spring Boot backend can handle when hitting the Vector Cache vs. when waiting on the LLM API bottleneck.
 - **6. Comparison between different sets of database!** (Minimum 3)
-
7. **Telemetry Logging** → As the system runs, it continuously logs latency metrics, hit/miss ratios, and token cost savings to generate experimental data for the research paper
-

Architecture / Workflow

1. **Client Request:** User asks a question via the frontend.
2. **API Gateway:** Request hits the Java Spring Boot backend deployed on AWS.
3. **Embedding Generation:** Backend converts the text query into a vector embedding using a lightweight, fast embedding model.
4. **Semantic Search (The Cache):** Backend queries the MongoDB Atlas Vector Database.
 - *Cache Hit (Similarity > 95%):* Return stored answer instantly.
 - *Cache Miss:* Forward to the heavy LLM, get the answer, serve to user, and save the new prompt/answer embedding pair to MongoDB for future use.

Known Problems? Not really a problem!

- Why prefer Semantic Caching? Standard Redis Caching → uses string matching
 - Latency in our project? → 20-50ms whereas LLM takes around 2-5seconds for generation!
-

Technical Problems with solution

⚠ Storage Efficiency

1. **Cost Weight Recency Algorithm** → LRU algorithm along with tokens comparison
→ Example: A Query used 5 second for processing and the other is a basic 50tkn reply
We can eliminate the older not recent and cheaper query from the DB.
2. **The Dusty Corner Method** → (K-Tail) → Instead of scanning all of the queries we can select the few of oldest available and not touched queries maybe a 100s or something!

⚠ Semanticity Problem → Now a 2 Step method!

1. **The Fast Sorting** → Fetching of the similar vectors without a lot of interpretation like `'password strength'` and `'password reset'`
→ Tossing them in a same space → **Intent missing but incredibly fast**
2. **Cross Encoding / Intent Reading** → Steps
 - 2.1. Retrieved Top 1 match's string is also extracted now
 - 2.2. String tuple is to be passed to lightweight cross encoders now

```
cross-encoder/ms-marco-MiniLM-L-6-v2 or  
cross-encoder/quora-distilroberta-base
```

2.3. The Model process the sequence and produce a score from [0,1]

2.4. Threshold of 0.85

- Ⓛ Latency is increased but how much? from 10-20 ms to sub 50ms
Still better than 2-5second processing of LLMs

3. The Real-World Vulnerabilities (Project Defense)

- **The Cache Stampede (The Waiting Room):** Multiple but same query → Coalescing mechanism solves it!
- **Cache Poisoning (The Gossip Effect):** Hallucination → Common Failure
Checking before adding to DB
- **Multi-Turn State Loss (The Goldfish Memory):** Relevancy → Passing of old history to fast sequence to sequence models like llama-8b-instruct